

بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department



CPCS-214 Computer Architecture and Organization

39 - Pipeline Hazards

Introduction

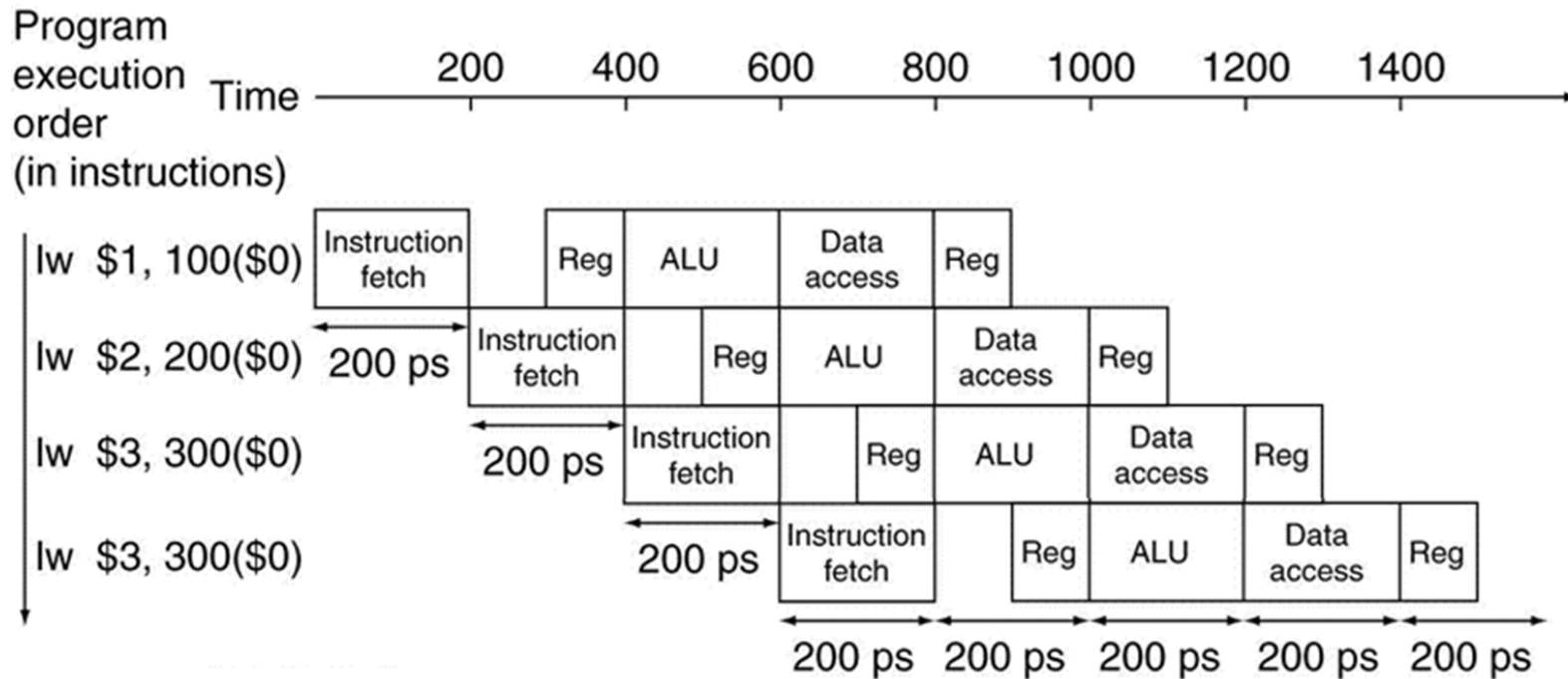
- Situations in pipelining when the next instruction cannot execute in the following clock cycle
- These events are called *hazards*
- Three different types
 - Structural
 - Data
 - Control

Structural Hazard

- Means that HW cannot support combination of instructions that we want to execute in the same clock cycle
- Carefully scheduled pipeline plans would then be foiled
- MIPS instruction set was designed to be pipelined
 - Making it easy for designers to avoid structural hazards
- Suppose that we had single memory instead of two memories

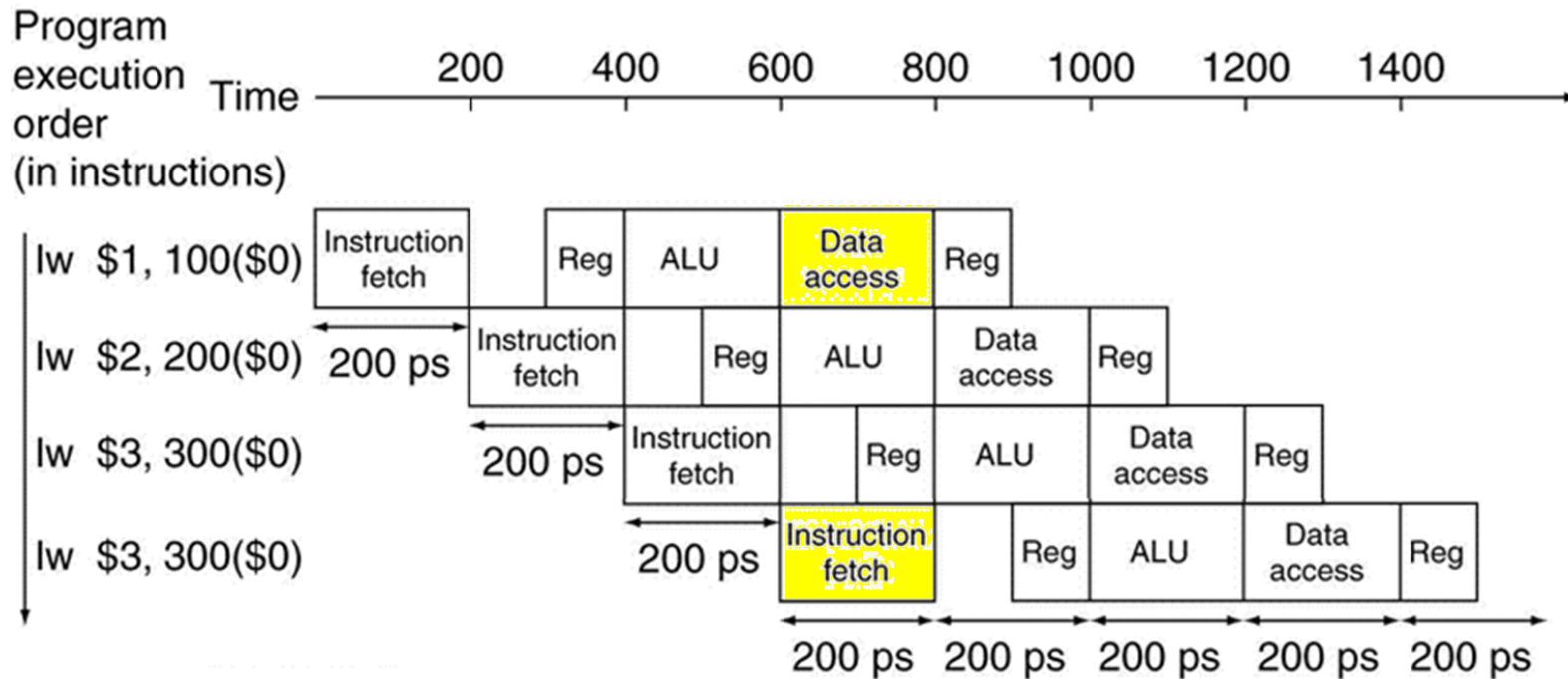
Structural Hazard

- Figure shows 4 instructions are executing in the pipeline



Structural Hazard

- Figure shows 4 instructions are executing in the pipeline



Structural Hazard

- In the same clock cycle
 - first instruction is accessing data from memory while
 - fourth instruction is fetching instruction from same memory
- Without two memories, pipeline could have a structural hazard

Data Hazard

- Occurs when pipeline must be stalled because one step must wait for another to complete
- In a computer pipeline, data hazards arise from the **dependence** of one instruction on an earlier one that is still in pipeline
- It could severely **stall pipeline**

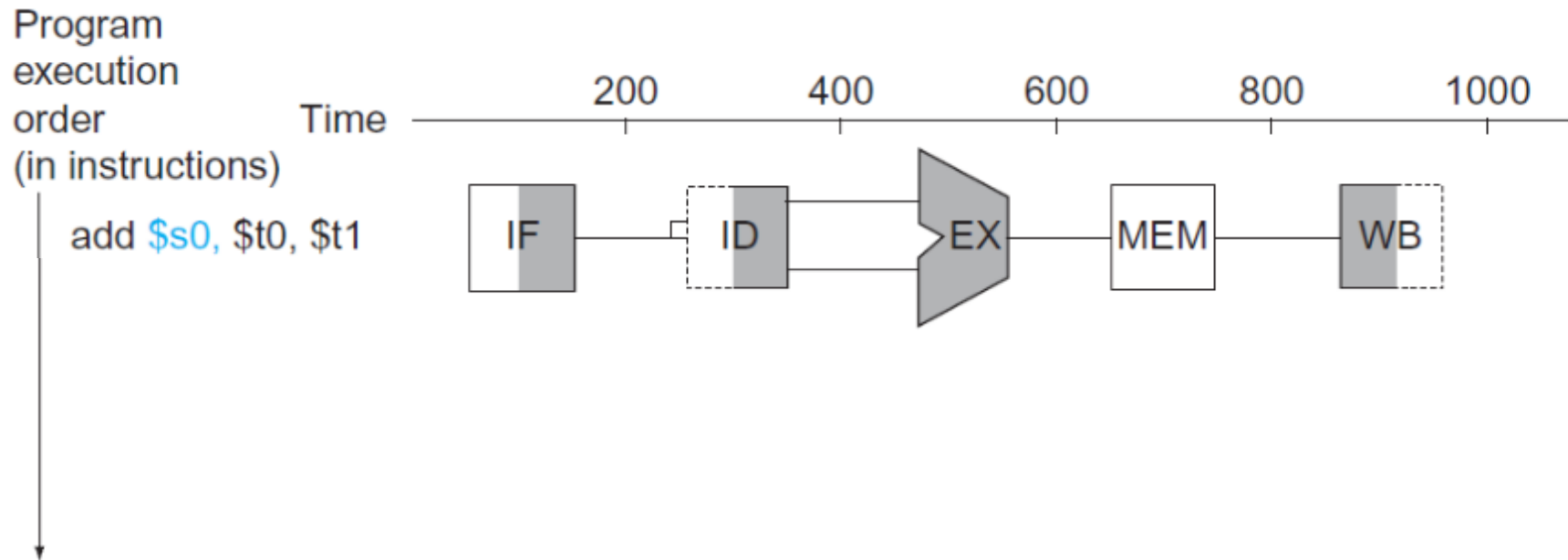
Data Hazard

- For example, **add** instruction is followed immediately by **subtract** instruction that uses the sum (\$s0):

```
add $s0, $t0, $t1  
sub $t2, $s0, $t3
```

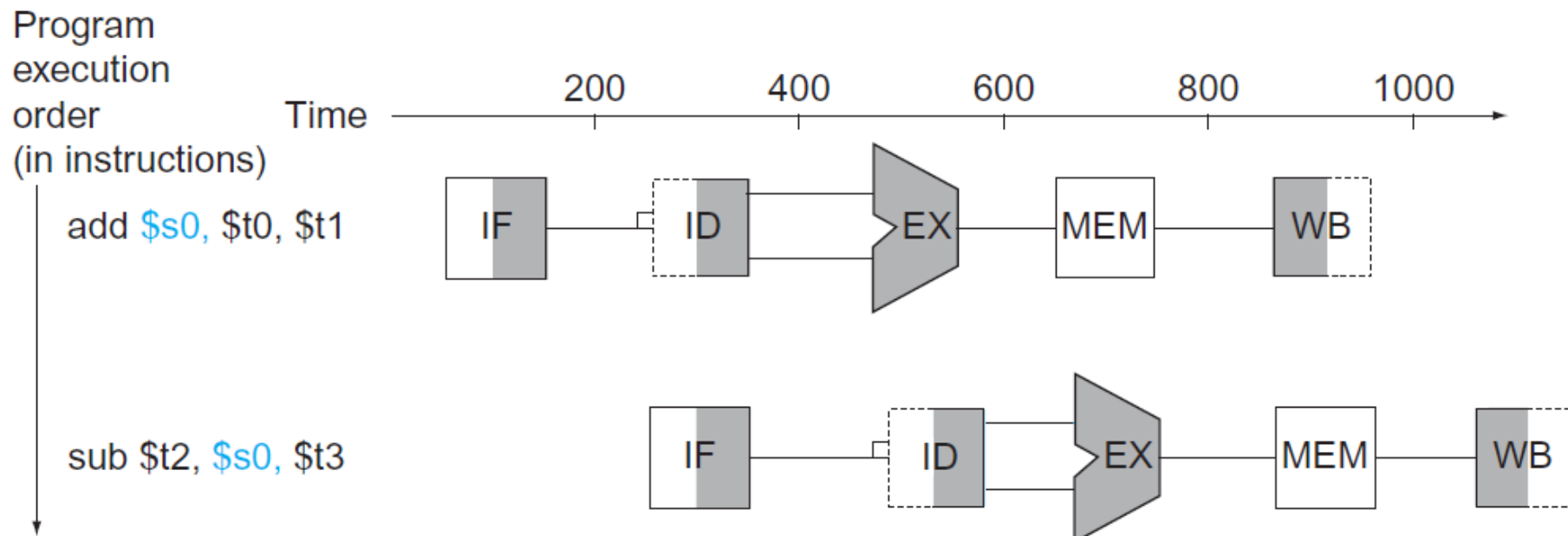

Data Hazard

- Add instruction doesn't write its result until the fifth stage



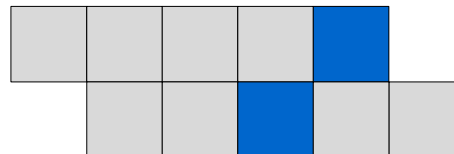
Data Hazard

- Add instruction doesn't write its result until the fifth stage
- Result is needed by sub instruction at the third stage



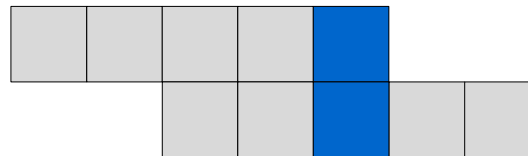
Delay

- Delay execution of second instruction for 3 cycles



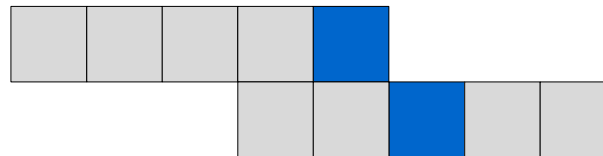
Delay

- Delay execution of second instruction for 3 cycles



Delay

- Delay execution of second instruction for 3 cycles
- Would have to waste 3 clock cycles in pipeline



Performance

- Although we could try to rely on compilers to remove all such hazards, results would not be satisfactory
 - Dependences happen just too often
 - Delay is just too long to expect compiler to rescue from this dilemma
- Solution to avoid delay?

Analogy

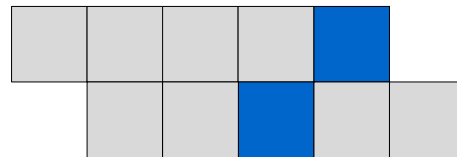
- Perform the following steps:
 1. Add two numbers using calculator.
 2. Record result on a paper.
 3. Copy result from the paper to the board.
- How to be faster

Analogy

- Perform the following steps:
 1. Add two numbers using calculator.
 2. Record result on a paper.
 3. Copy result from the paper to the board.
- How to be faster
 - Step 3
 - no need to wait to copy result from paper to board
 - copy directly from calculator to board (forwarding)

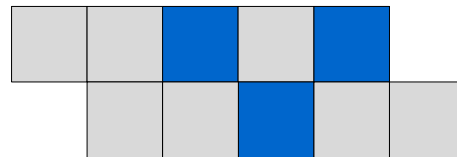
Forwarding

- Don't need to wait for instruction to complete before trying to resolve data hazard



Forwarding

- Don't need to wait for instruction to complete before trying to resolve data hazard
- For the code sequence, as soon as ALU creates sum for add, supply it as an input for subtract
- Adding extra HW to retrieve missing item early from internal resources is called **forwarding** or **bypassing**



Example

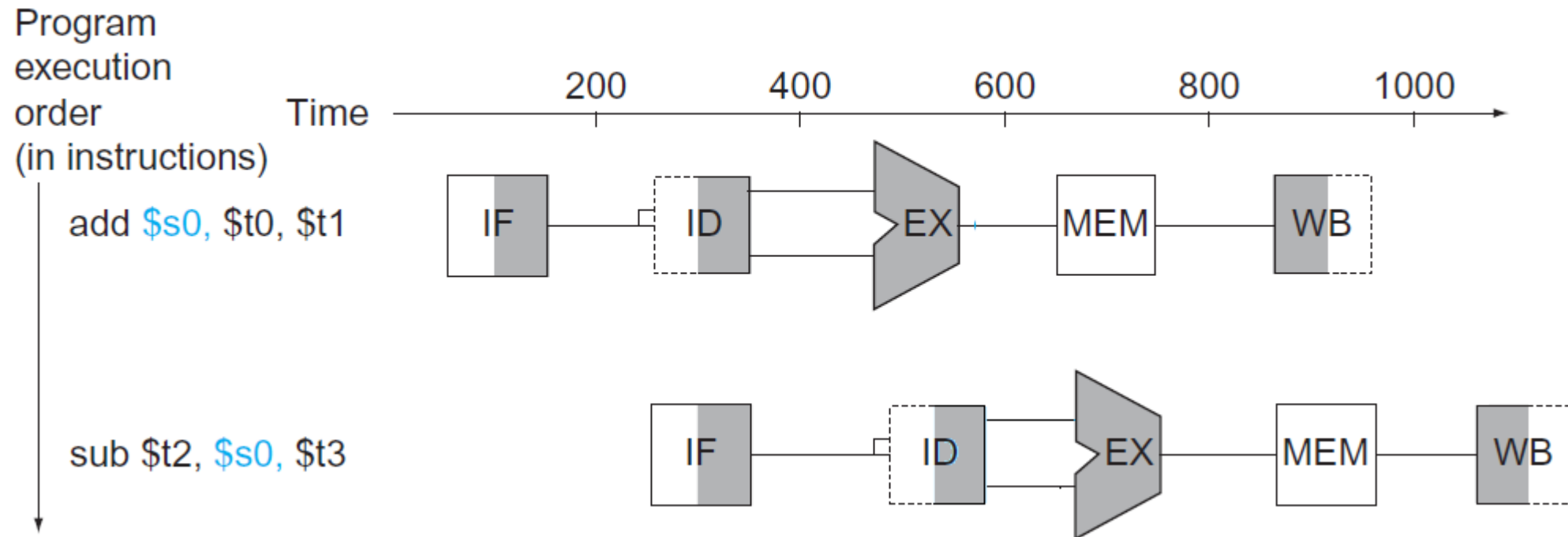
- For the two instructions, show what pipeline stages would be connected by forwarding.

```
add $s0, $t0, $t1  
sub $t2, $s0, $t3
```

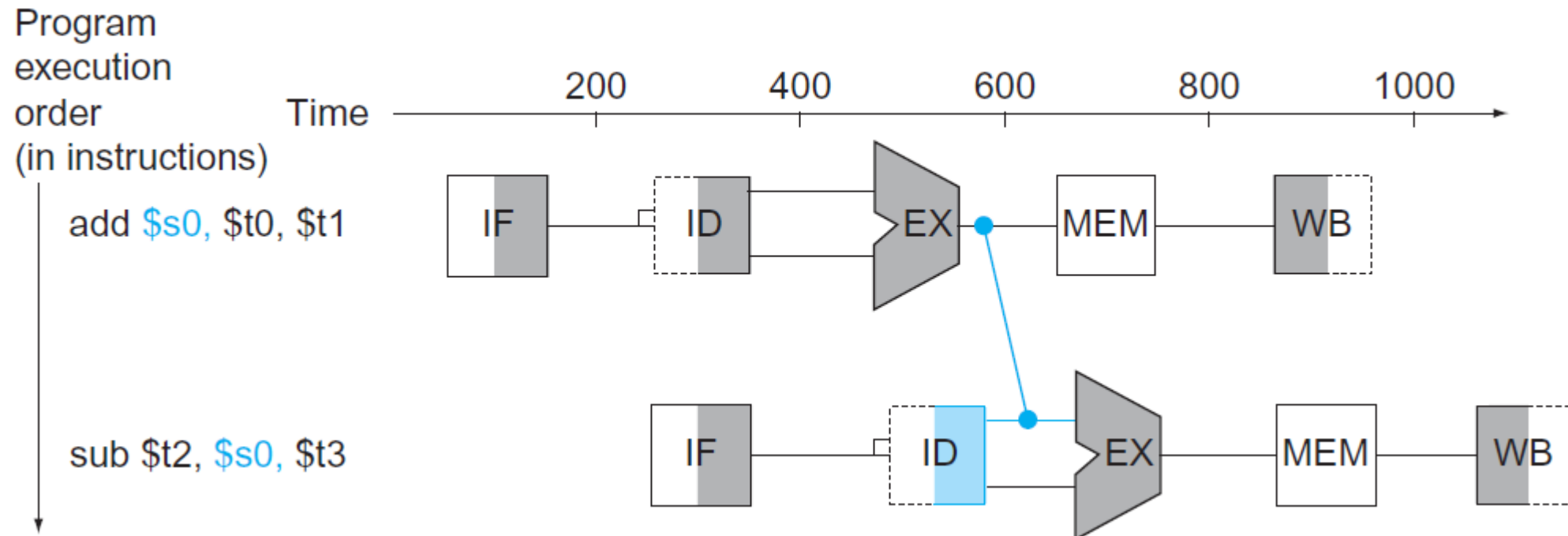
Example (cont.)

- Use drawing to represent datapath during stages of pipeline
- Align a copy of datapath for each instruction
- Figure shows **connection to forward** value in \$s0 after execution stage of *add* instruction as input to execution stage of *sub* instruction

Example (cont.)



Example (cont.)



Forwarding Backward

- Forwarding paths are **valid** only if destination stage is later in time than source stage
- **Cannot be a valid** forwarding path from the output of the memory access stage in first instruction to input of execution stage of following
 - Since that would mean going **backward** in time

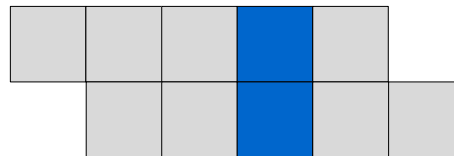
Load-Use Data Hazard

- Forwarding works very well
- However, it cannot prevent all pipeline stalls
- For example, suppose first instruction is a *load* instead of *add*

```
lw $s0, 20($t1)
sub $t2, $s0, $t3
```

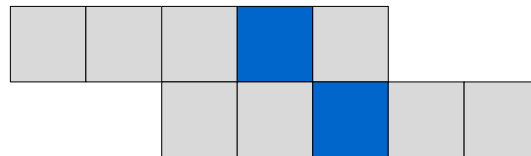

Load-Use Data Hazard

- Desired data would be available only after fourth stage of first instruction in the dependence, which is too late for the *input* of the third stage of sub
- Forwarding

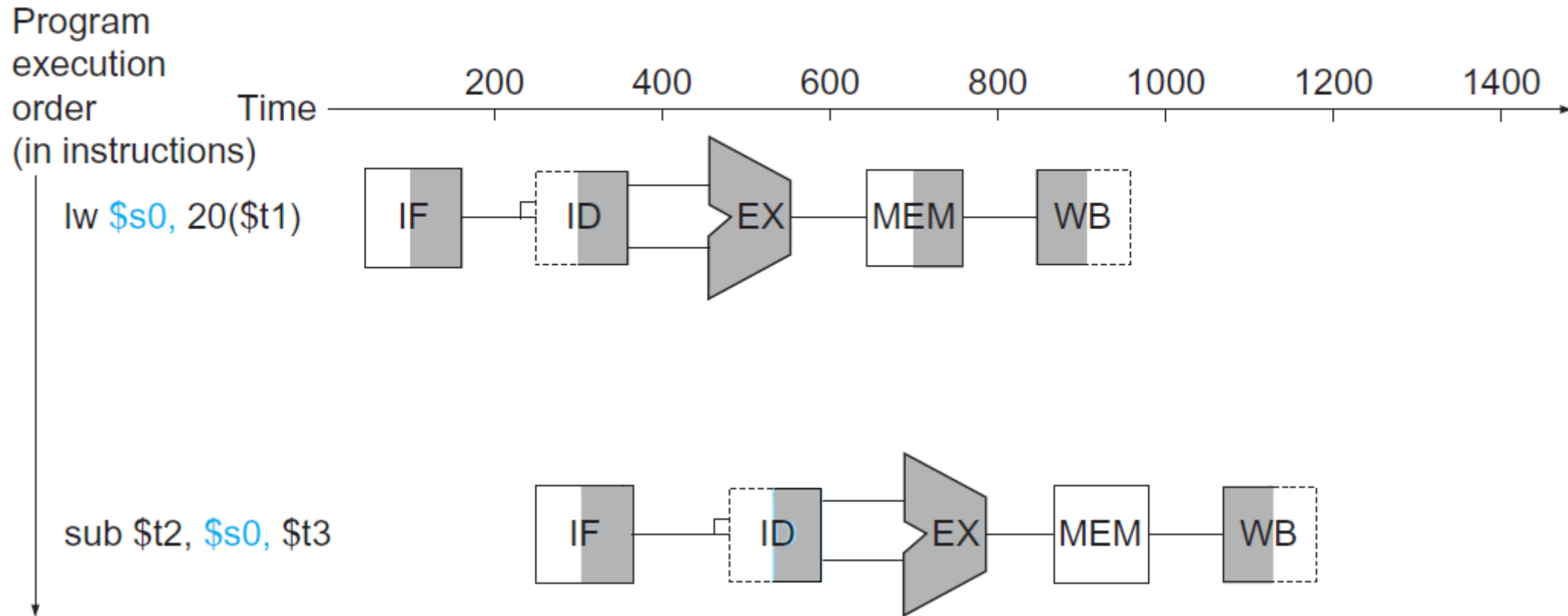


Load-Use Data Hazard

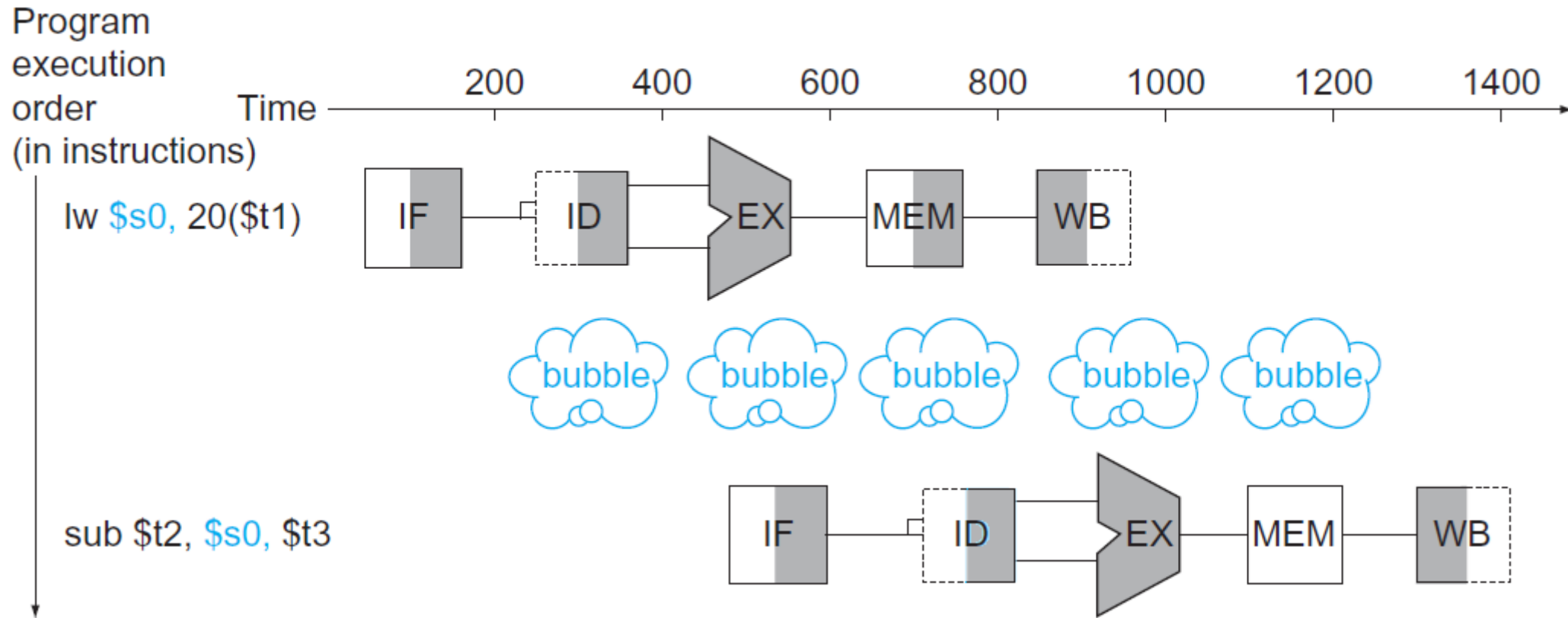
- Desired data would be available only after fourth stage of first instruction in the dependence, which is too late for the *input* of the third stage of sub
- Hence, even with forwarding, we would have to stall one stage for a load-use data hazard



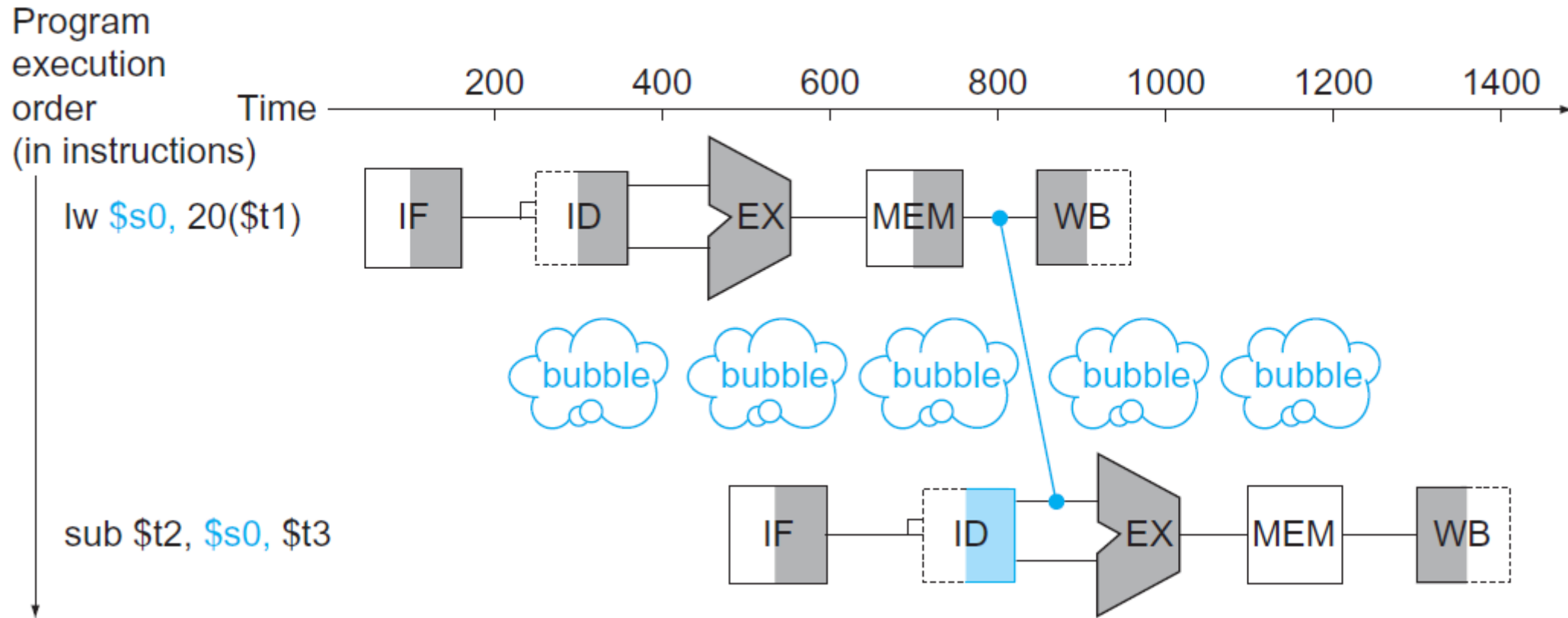
Load-Use Data Hazard



Stall



Stall and Forwarding



Pipeline Stall

- Important pipeline concept, called **pipeline stall**
 - Often given nickname **bubble**
- Shall see stalls elsewhere in the pipeline
- To handle hard cases like these, using either
 - HW detection and stalls or
 - SW that reorders code to try to avoid load-use pipeline stalls

Reordering Code to Avoid Pipeline Stalls

- In some cases code would be reordered to utilize stall cycle with independent instruction

Example

- Consider the following code segment in C/Java:
 $a = b + e;$
 $c = b + f;$
- Assuming all variables are in memory and are addressable as offsets from \$t0. Find the hazards in the generated code segment and reorder the instructions to avoid any pipeline stalls.

Example (cont.)

- Generated MIPS code:

```
lw $t1, 0($t0)
lw $t2, 4($t0)
add $t3, $t1, $t2
sw $t3, 12($t0)
lw $t4, 8($t0)
add $t5, $t1, $t4
sw $t5, 16($t0)
```

Example (cont.)

- Generated MIPS code:

```
lw $t1, 0($t0)
```

```
lw $t2, 4($t0)
```

```
add $t3, $t1, $t2
```

```
sw $t3, 12($t0)
```

```
lw $t4, 8($t0)
```

```
add $t5, $t1, $t4
```

```
sw $t5, 16($t0)
```

Example (cont.)

- Both `add` instructions have a hazard because of their respective dependence on immediately preceding `lw` instruction
- Bypassing eliminates several other potential hazards, including dependence of first `add` on first `lw` and any hazards for store instructions

Example (cont.)

- Generated MIPS code:

```
lw $t1, 0($t0)
lw $t2, 4($t0)
stall → add $t3, $t1, $t2
sw $t3, 12($t0)
stall → lw $t4, 8($t0)
add $t5, $t1, $t4
sw $t5, 16($t0)
```

Example (cont.)

- Generated MIPS code:

```
lw $t1, 0($t0)
```

```
lw $t2, 4($t0)
```

```
add $t3, $t1, $t2
```

```
sw $t3, 12($t0)
```

```
lw $t4, 8($t0)
```

```
add $t5, $t1, $t4
```

```
sw $t5, 16($t0)
```

- Reordered MIPS code:

Example (cont.)

- Generated MIPS code:

```
lw $t1, 0($t0)
lw $t2, 4($t0)
add $t3, $t1, $t2
sw $t3, 12($t0)
lw $t4, 8($t0)
add $t5, $t1, $t4
sw $t5, 16($t0)
```

- Reordered MIPS code:

```
lw $t1, 0($t0)
lw $t2, 4($t0)
lw $t4, 8($t0)
add $t3, $t1, $t2
sw $t3, 12($t0)
add $t5, $t1, $t4
sw $t5, 16($t0)
```

Performance

- On a pipelined processor with forwarding, the reordered sequence will complete in two fewer cycles than the original version

Control Hazards

- Arising from need to make decision based on the results of one instruction while others are executing
- Equivalent decision task in a computer is **branch** instruction

Branch

- Must begin fetching instruction following the branch on the next clock cycle
- Pipeline cannot possibly know what next instruction should be, since it *only just received* the branch instruction from memory!

```
add $4, $5, $6  
beq $1, $2, 40  
lw $3, 300($0)
```

```
40: or $7, $8, $9
```

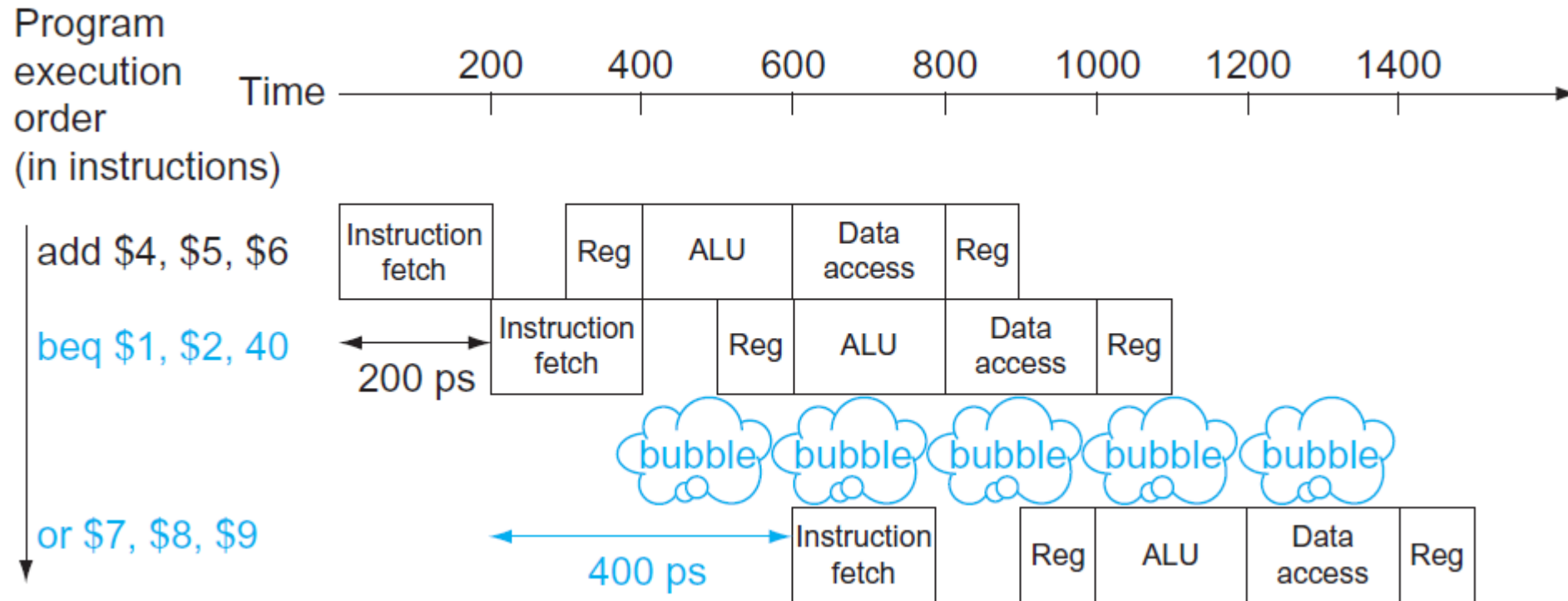
Possible Solution

- **Stall** immediately after fetch a branch
- Waiting until pipeline determines outcome of the branch and knows what instruction address to fetch from

Stalling on Branch

- Put in enough extra HW so that it can test registers, calculate branch address, and update PC during second stage of pipeline
- Even with this extra HW, pipeline involving conditional branches would look like next figure
- **lw** instruction, executed if branch fails, is stalled one extra 200 ps clock cycle before starting

Stalling on Branch



Performance of Stall on Branch

- Estimate impact on CPI of stalling on branches. Assume all other instructions have a CPI of 1, and branches are 17% of the instructions executed.
- Since other instructions run have a CPI of 1, and branches took one extra clock cycle for stall, then CPI is 1.17 and hence slowdown of 1.17 versus ideal case

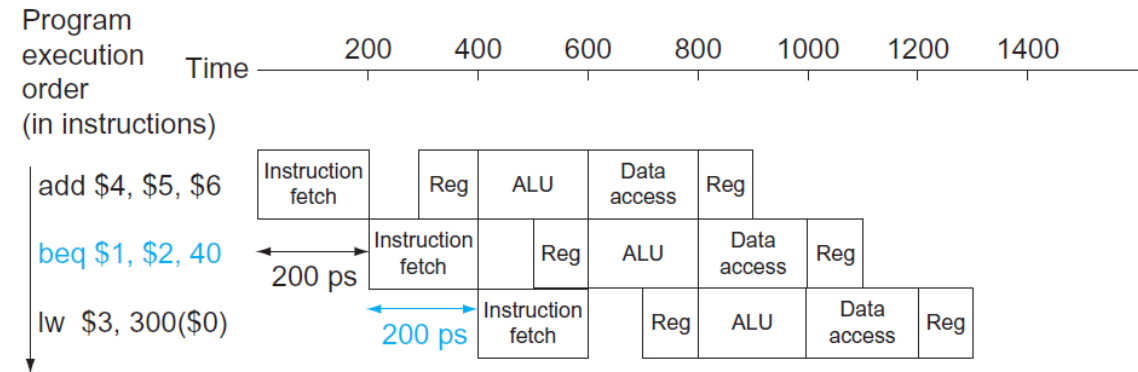
Performance of Stall on Branch

- If we cannot resolve branch in the second stage, as is often the case for longer pipelines, then we'd see an even larger slowdown if we stall on branches
- Cost of this option is too high for most computers to use
- Motivates a second solution to control hazard using one of the great ideas

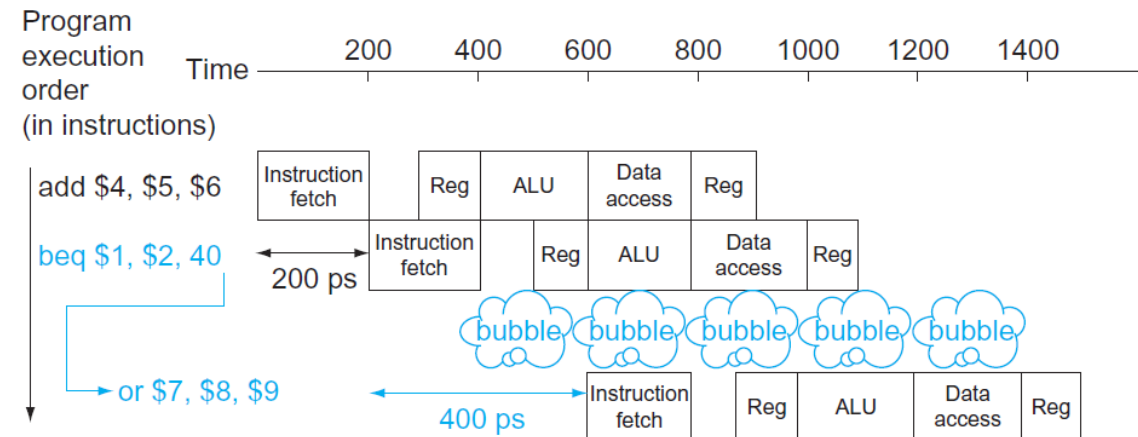
Prediction

- Computers do indeed use **prediction** to handle branches
- One approach is to predict that branches will be **untaken**
 - When **right**, pipeline proceeds at **full speed**
 - Only when branches are **taken** does the pipeline **stall**
- This option does not slow down pipeline when **correct**
- When **wrong**, however, need to redo the task that has been done while guessing the decision

Predicting Branch Not Taken



Correct



Incorrect

Backward Prediction

- A more sophisticated version of branch prediction would have some branches predicted as taken and some as untaken
- In programming
 - At bottom of loops are branches that jump back to top of loop
 - Since they are likely to be taken and they branch backward, we could always predict taken for branches that jump to an earlier address
- Such rigid approaches rely on stereotypical behavior and don't account for individuality of a specific branch instruction

Dynamic Prediction

- *Dynamic* HW predictors
- Make their guesses depending on behavior of each branch and may change predictions for a branch over life of a program
 - Adjusting next *prediction* depending on success of recent guesses

Prediction History

- One popular approach to dynamic prediction of branches is keeping a **history** for each branch as **taken** or **untaken**
- Then using the recent past behavior to predict the future
- Can correctly predict branches with +90% accuracy
- When guess is wrong
 - Pipeline control must ensure that instructions following wrongly guessed branch have no effect and
 - must restart pipeline from the proper branch address

Long Pipeline

- As in the case of all other solutions to control hazards, longer pipelines exacerbate the problem, in this case by raising the **cost of misprediction**

Delayed Branch

- A third approach to control hazard
- Solution used by MIPS architecture
- Delayed branch always executes next sequential instruction, with the branch taking place *after* that one instruction delay
 - Hidden from programmer because assembler can automatically arrange instructions to get branch behavior desired by programmer

Delayed Branch

- MIPS SW will place an instruction immediately after the delayed branch instruction that is not affected by branch, and a taken branch changes address of instruction that *follows* this safe instruction
 - In the example, `add` instruction before branch does not affect branch and can be moved after branch to fully hide branch delay

```
add $4, $5, $6
beq $1, $2, 40
lw $3, 300($0)
```

```
40: or $7, $8, $9
```

Delayed Branch

- Since delayed branches are useful when branches are short, no processor uses a delayed branch of more than one cycle
- For longer branch delays, HW-based branch prediction is usually used

Pipeline Overview Summary

- Technique that exploits **parallelism** among instructions in a sequential instruction stream
- Substantial advantage
 - Unlike programming a multiprocessor, it is **invisible** to programmer